Django

Capítulo 3 Trabalhando com models

IFRN
George Azevedo

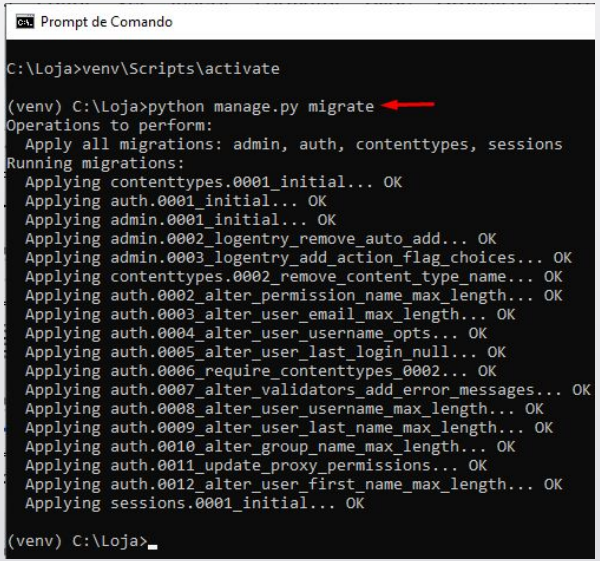
Django
Capítulo 3 - Trabalhando com Models

Models são classes responsáveis por representar nossas tabelas do banco de dados dentro de nossa aplicação. É através dele que interagimos com o banco de dados, lendo, criando, alterando e/ou deletando registros. No Django, models viram páginas no painel administrativos onde poderemos também interagir com o banco de dados. Usaremos o banco de dados padrão do Django, o SQLite. Então poderemos baixar o SQLite Studio para poder ver a estrutura SQL do banco de dados.

Django
Capítulo 3 - Trabalhando com Models

Antes de configurar models precisamos rodar nossa primeira migração, para criar as tabelas padrões de uma aplicação django. para isso usamos o comando, **com o ambiente virtual ativo**, no cmd:

```
python manage.py migrate
```



Django
Capítulo 3 - Trabalhando com Models

Com essa estruturação inicial feita, podemos criar um usuário superadmin que terá acesso total a nossa aplicação. O comando que usaremos irá mandar um formulário, em linha de comando, pedindo primeiro o nome, depois o email, a senha, por fim pedirá uma confirmação para criar o superadmin com y(para sim) e N(para não), então, quando for perguntado, escolha sim. Em ambiente de produção, é recomendável a senha ser segura, mas como estamos apenas aprendendo, algo simples e que seja fácil de você lembrar é recomendável. Use o comando

```
python manage.py createsuperuser
```

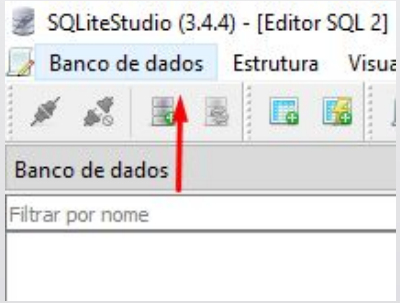
Django
Capítulo 3 - Trabalhando com Models

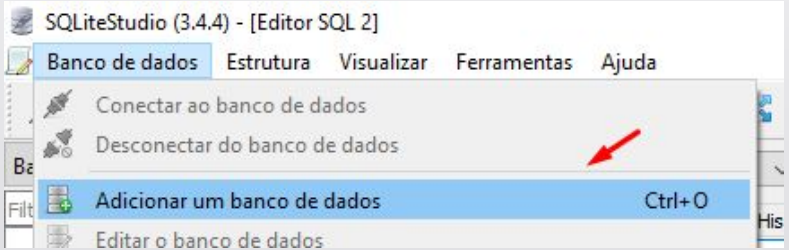
```
(venv) C:\Loja>python manage.py createsuperuser
Usuário (leave blank to use 'thiagogonsalves'): admin
Endereço de email: admin@gmail.com
Password:
Password (again):
A senha é muito parecida com usuário
Esta senha é muito curta. Ela precisa conter pelo menos 8 caracteres.
Esta senha é muito comum.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.

(venv) C:\Loja>
```

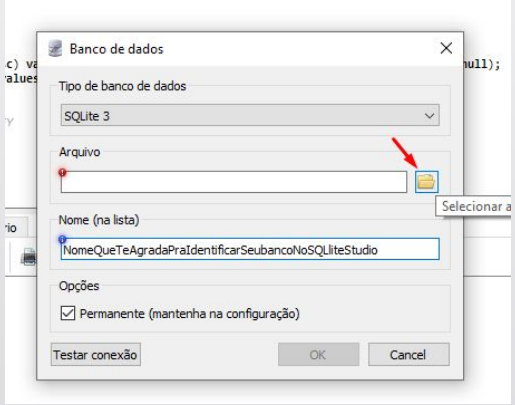
A seta vermelha mostra o nome do meu usuário, a seta azul mostra meu email, as verdes mostram, respectivamente, minha senha e minha senha digitada novamente, ambas sendo ocultas. Por minha senha ser muito simples(“admin”), a interface de comando mandou avisos antes da confirmação(seta laranja).

Django
Capítulo 3 - Trabalhando com Models

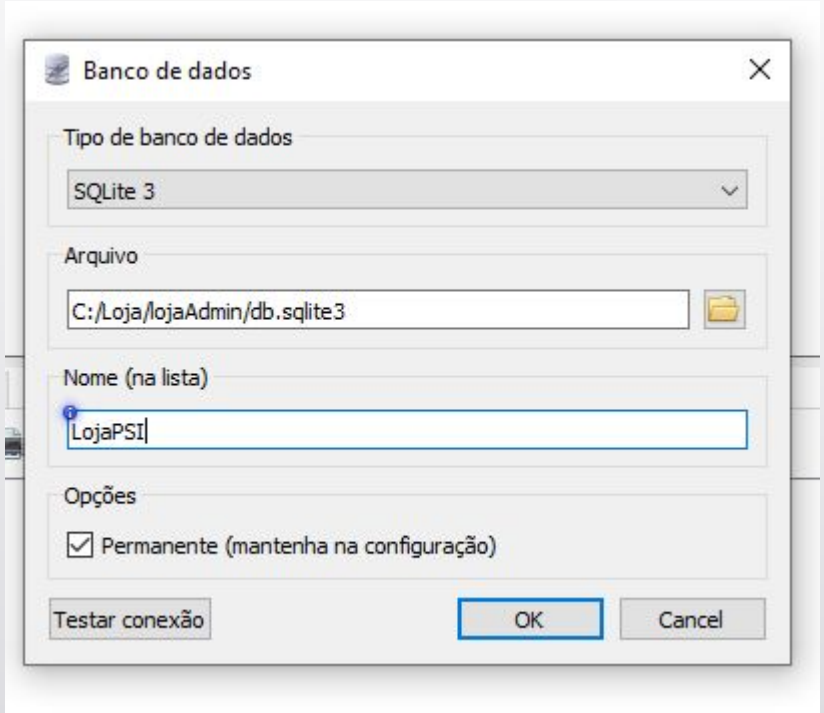
Com isso nossa aplicação já possuirá um banco de dados padrão do Django e usuário que pode acessar tudo nele. Esse banco está no arquivo db.sqlite3. Se já deseja ver a estrutura basta ir no SQLite Studio e ir na opção Banco de dados, depois adicionar banco, aperta na pasta amarela , procurar o arquivo db.sqlite3 e escolher ele.

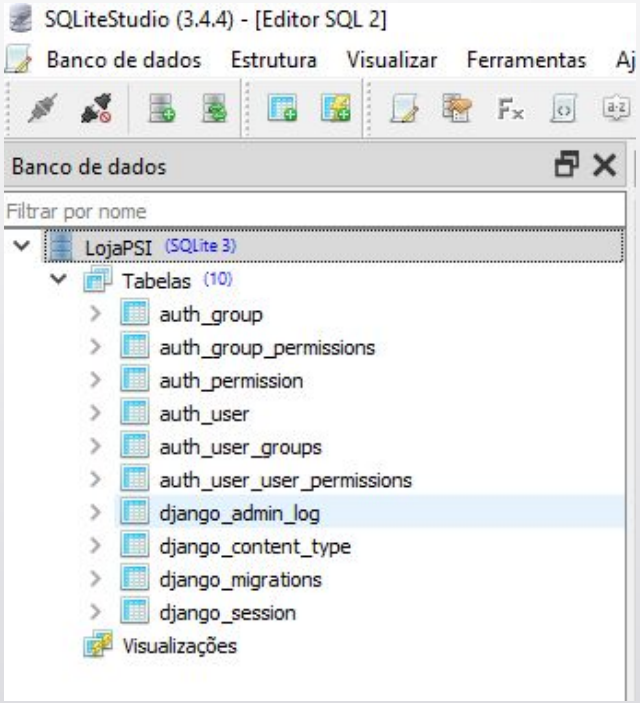






Django
Capítulo 3 - Trabalhando com Models



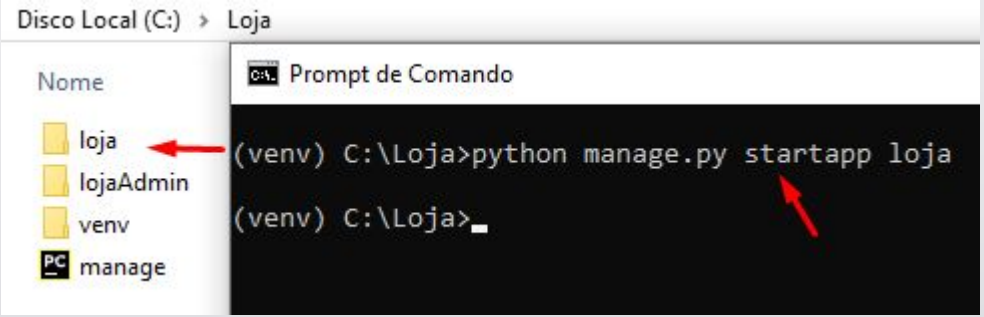


Django
Capítulo 3 - Trabalhando com Models

O Django possui o conceito de app, onde cada app dentro do sistema tem sua finalidade específica. Para criarmos nosso primeiro app executaremos o seguinte comando no cmd :

```
python manage.py startapp loja
```

OBS: lembrando que o comando deve ser executado na mesma pasta e que lojaAdmin está pois esse comando cria uma estrutura no diretorio em que você está localizado no cmd.



Django
Capítulo 3 - Trabalhando com Models

Agora precisamos adicionar esse app para ser gerenciado por nosso admin, nosso projeto Django, lojaAdmin. Modificando a lista de INSTALLED_APPS no arquivo lojaAdmin/settings/**settings.py**, adicionando uma vírgula depois 'django.contrib.staticfiles' e após a vírgula adicionamos 'loja' assim:

```
31
32  # Application definition
33
34  INSTALLED_APPS = [
35      'django.contrib.admin',
36      'django.contrib.auth',
37      'django.contrib.contenttypes',
38      'django.contrib.sessions',
39      'django.contrib.messages',
40      'django.contrib.staticfiles',
41      'loja'
42  ]
43
```

Django
Capítulo 3 - Trabalhando com Models

Agora, no app criado, **adicione uma pasta chamada models** e apague o arquivo models.py, para organizar em múltiplos arquivos as classes, ao invés de só organizar tudo em um só arquivo. Depois crie um arquivo chamado __init__.py.

Django
Capítulo 3 - Trabalhando com Models

Agora, com a pasta models criado, criaremos nossa primeira classe modelo, chamada Fabricante com o seguinte código:
loja/models/**Fabricante.py**:

```
from loja.models import *
class Fabricante(models.Model):
    Fabricante = models.CharField(null=False, max_length=100)
    criado_em = models.DateTimeField(auto_now_add=True)
    alterado_em = models.DateTimeField(auto_now=True)

    def __str__(self):
        return '{}'.format(self. Fabricante)
```

Django
Capítulo 3 - Trabalhando com Models

O Atributo **Fabricante** armazena o nome do fabricante. “Models.CharField” é um tipo de modelo que o Django oferece, que possui um input do tipo texto de até 255 caracteres associado a ele. O parâmetro “null=False” indica que esse valor não pode ser nulo e “max_length=100” determina o número máximo de caracteres,100.

O **criado_em** armazena a data em que o objeto da classe foi adicionado.

O **alterado_em** armazena a data da última modificação do objeto.

O **metodo __str__** retorna o nome do objeto Fabricante.

Django
Capítulo 3 - Trabalhando com Models

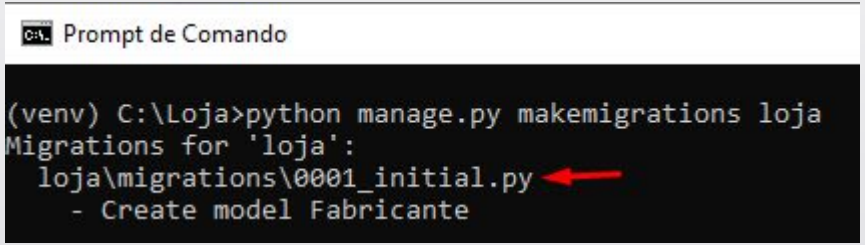
Agora precisamos comunicar ao Django quais models deverão refletir a estrutura de nosso banco. Para isso no init, que criamos dentro da pasta models do nosso app criado, colocaremos o seguinte código para montar nossa tabela:

```
    from django.db import models
    from django.contrib.auth.models import User
    from django.db.models.signals import post_save
    from django.dispatch    import  receiver
    #acima são bibliotecas padrões necessárias do Django, e abaixo
nossos models
    from .Fabricante import Fabricante
```

Django
Capítulo 3 - Trabalhando com Models

Agora, basta ir no cmd e digitar o comando a seguir, para criar um código que conterá a estrutura de nossa tabela com base no nosso modelo:

```
python manage.py makemigrations loja
```



<u>OBS</u>: a seta está apontando pro nome do arquivo que conterá o código citado, e a linha que mostra sua localização no projeto.

Django
Capítulo 3 - Trabalhando com Models

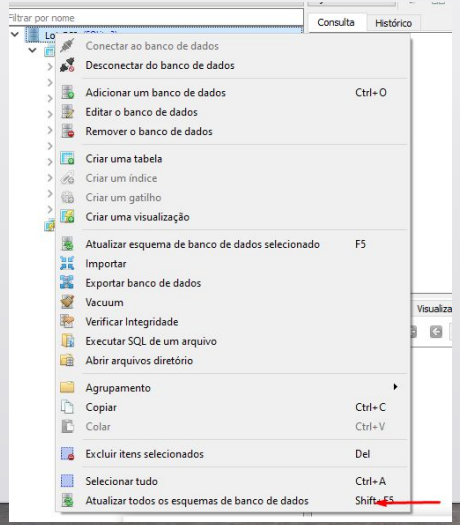
Depois disso será criado um arquivo na pasta migration do app com o código da implementação do nosso model. Agora, para de fato aplicarmos essa alteração basta usar o comando:

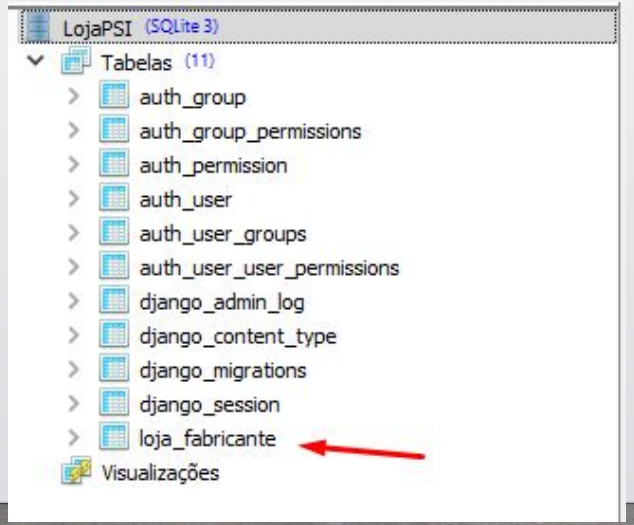
```
python manage.py migrate loja
```

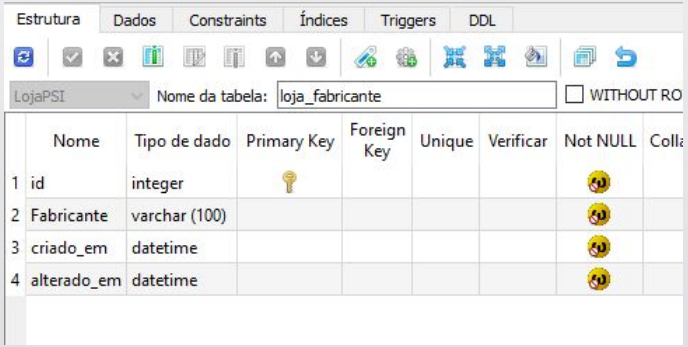
```
(venv) C:\Loja>python manage.py migrate loja
Operations to perform:
  Apply all migrations: loja
Running migrations:
  Applying loja.0001_initial... OK

(venv) C:\Loja>
```

Django
Capítulo 3 - Trabalhando com Models

Para conferir a nova tabela basta ir no SQLite Studio e clicar com o botão direito no banco de dados e clicar em atualizar todos os esquemas de banco de dados clique dias vezes na tabela e veja sua estrutura.







Django
Capítulo 3 - Trabalhando com Models

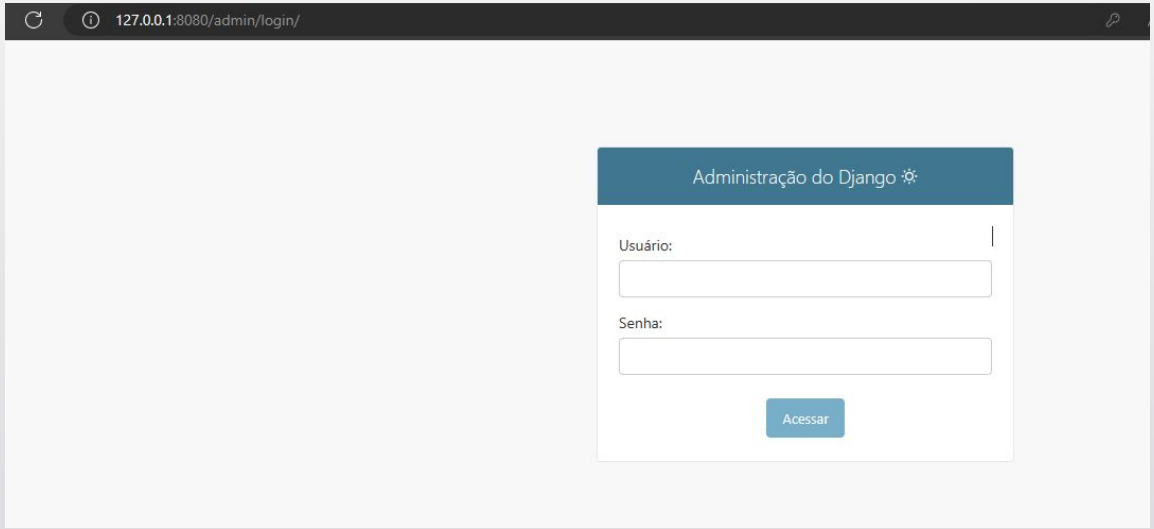
Com o django temos acesso a manipulações de registro com interface gráfica, para isso precisamos rodar a aplicação, e colocar no url do navegador http://127.0.0.1:8080/admin. Ao entrarmos nesse link, veremos um formulário de entrada com login. Por hora, o único usuário com acesso a entrar é nosso superadmin. Então preencha o formulário com o nome e senha do seu superadmin. Por agora rode o projeto com o seguinte comando no cmd:

```
python manage.py runserver 127.0.0.1:8080
```

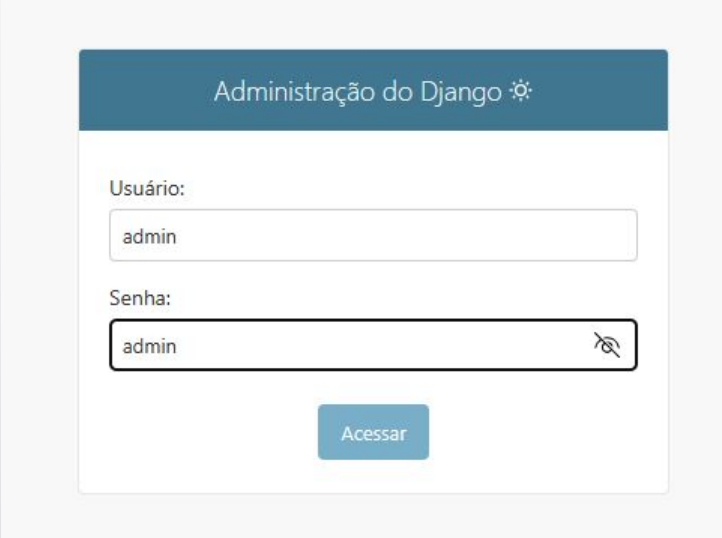
```
(venv) C:\Loja>python manage.py runserver 127.0.0.1:8080
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
January 15, 2024 - 01:39:47
Django version 4.2.7, using settings 'lojaAdmin.settings.development'
Starting development server at http://127.0.0.1:8080/
Quit the server with CTRL-BREAK.
```

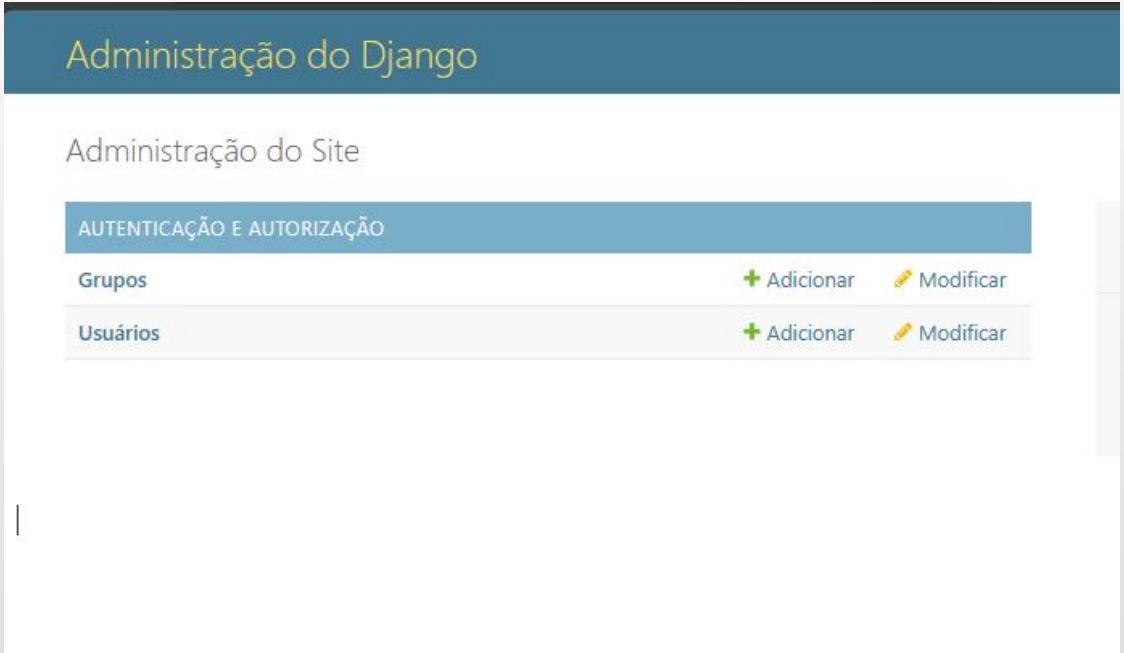
Django
Capítulo 3 - Trabalhando com Models

Entre no link http://127.0.0.1:8080/admin e sera exibido o seguinte formulário. Preencha com os dados do super admin e submeta para ver as tabelas disponíveis para ele:



Django
Capítulo 3 - Trabalhando com Models





Django
Capítulo 3 - Trabalhando com Models

O que fizemos até agora foi adicionar o nosso modelo a uma tabela do nosso banco, porém não adicionamos essa tabela na interface do admin, por isso não a vemos na interface. Para isso devemos, na pasta do nosso app criado, no arquivo admin.py(loja/**admin.py**), adicionar o nosso modelo na interface assim:

```
from django.contrib import admin #isso já vai estar existindo no
arquivo
# Register your models here.
from .models import * #imporata nossos models
admin.site.register(Fabricante)  #adiciona a interface do adm
```

Django
Capítulo 3 - Trabalhando com Models

Com isso, se rodarmos nosso projeto de novo e irmos na interface do admin já veremos a tabela de Fabricantes e poderemos listar, adicionar, deletar, ver ou/e modificar registros.

Django
Capítulo 3 - Trabalhando com Models

Antes de adicionarmos registros, vamos criar mais duas classes, Categoria e fabricante na pasta models com o seguinte conteúdo:

```
# loja/models/Categoria:
from loja.models import *
class Categoria(models.Model):
    Categoria = models.CharField(null=False, max_length=100)
    criado_em = models.DateTimeField(auto_now_add=True)
    alterado_em = models.DateTimeField(auto_now=True)

    def __str__(self):
        return '{}'.format(self.Categoria)
```

Django
Capítulo 3 - Trabalhando com Models

```
#loja/models/Produto.py parte 1:
from loja.models import *
class Produto(models.Model):
    Produto = models.CharField(null=False, max_length=100)
    destaque = models.BooleanField(default=True)
    promocao = models.BooleanField(default=True)
    msgPromocao = models.CharField(null=True, max_length=100)
    preco = models.DecimalField(max_digits=8, decimal_places=2)
    categoria = models.ForeignKey(Categoria, null=True,
related_name='categoria', on_delete=models.SET_NULL)
    fabricante = models.ForeignKey(Fabricante, null=True,
related_name='fabricante', on_delete=models.SET_NULL)
```

Django
Capítulo 3 - Trabalhando com Models

```
#loja/models/Produto.py parte 2:
    criado_em = models.DateTimeField(auto_now_add=True)
    alterado_em = models.DateTimeField(auto_now=True)

    def __str__(self):
        return '{}'.format(self.Produto)
```

Django
Capítulo 3 - Trabalhando com Models

“BooleanField” associa a variável ao valor True ou False. Também associa o atributo a um checkbox marcavel. O parâmetro “default=True” coloca um valor inicial que se manterá caso o usuário não interage com esse campo, no caso, por padrão sempre será True.

“DecimalFleld” associa a variável a valores reais e a um campo numérico, input type number.

“ForeignKey” serve para associar essa classe a outra, mais especificamente, para pegar as chaves primárias de objetos de outra tabela.

Django
Capítulo 3 - Trabalhando com Models

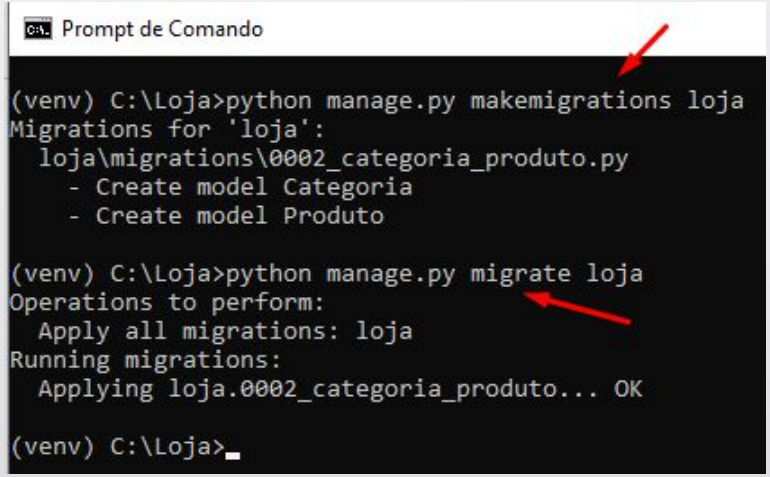
Novamente iremos ao models/**__init__.py** para que nossos novos models façam parte do nosso banco de dados:

```
from django.db import models
from django.contrib.auth.models import User
from django.db.models.signals import post_save
from django.dispatch import receiver
#acima são bibliotecas padrões necessárias do Django, e abaixo nossos
models
from .Fabricante import Fabricante
from .Categoria import Categoria
from .Produto import Produto
```

Django
Capítulo 3 - Trabalhando com Models

Agora vamos usar o comando para gerar o código da modificação no banco de dados e depois o comando para implementar a modificação desse arquivo no cmd:

```
python manage.py makemigrations loja
python manage.py migrate loja
```



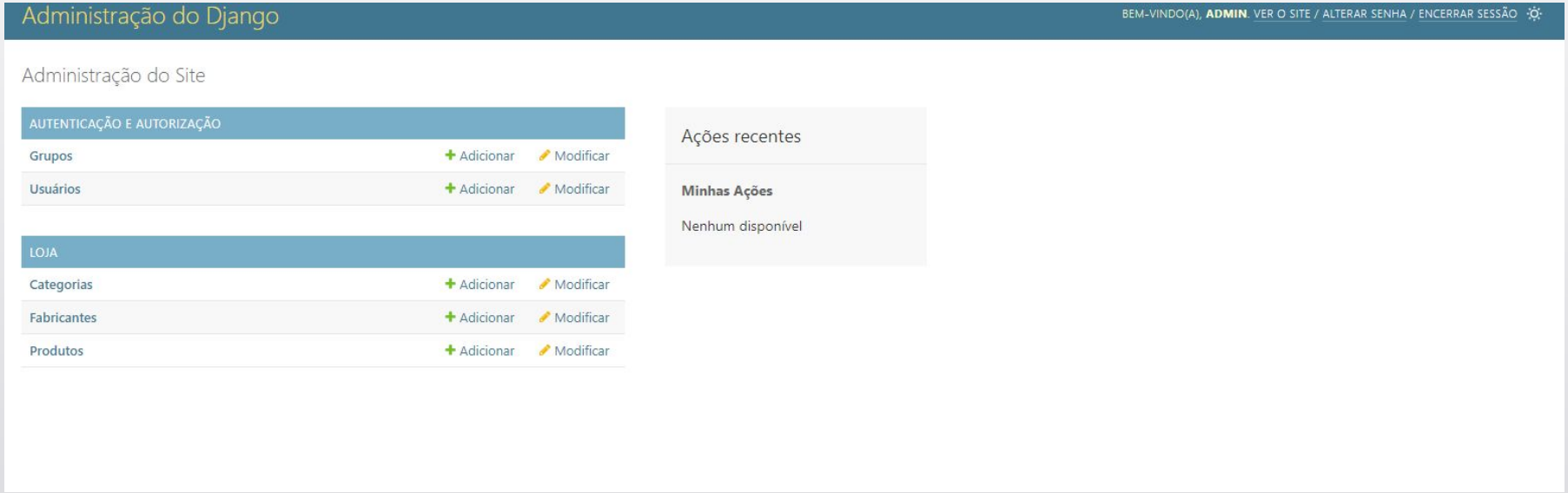
Django
Capítulo 3 - Trabalhando com Models

Após isso iremos no admin.py do nosso app criado(loja/**admin.py**) e adicionaremos as classes de modelos que criamos na interface do adm assim :

```
from django.contrib import admin
# Register your models here.
from .models import *
admin.site.register(Fabricante)
admin.site.register(Categoria)
admin.site.register(Produto)
```

Django
Capítulo 3 - Trabalhando com Models

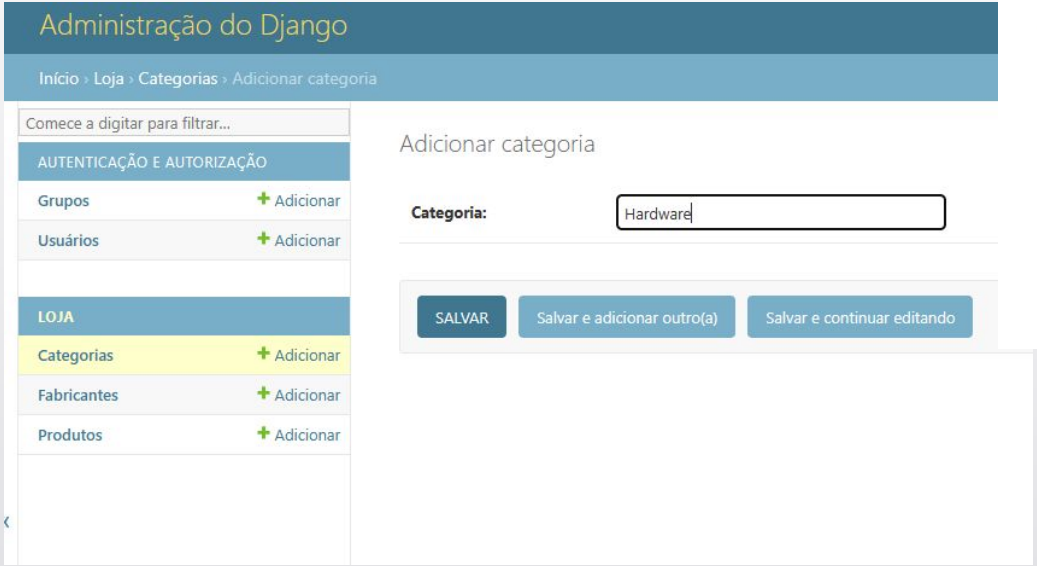
Agora se visitarmos a interface do admin de novo, poderemos ver as novas tabelas



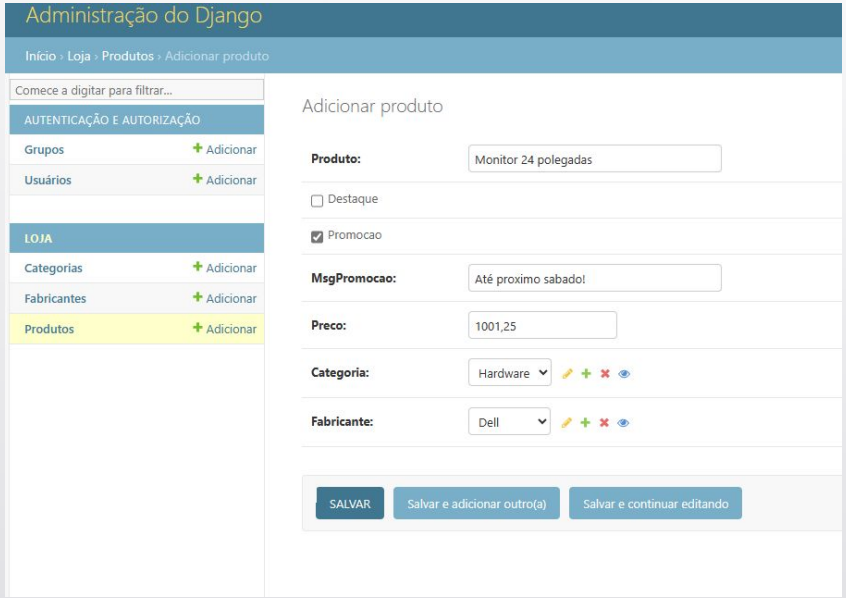
Django
Capítulo 3 - Trabalhando com Models

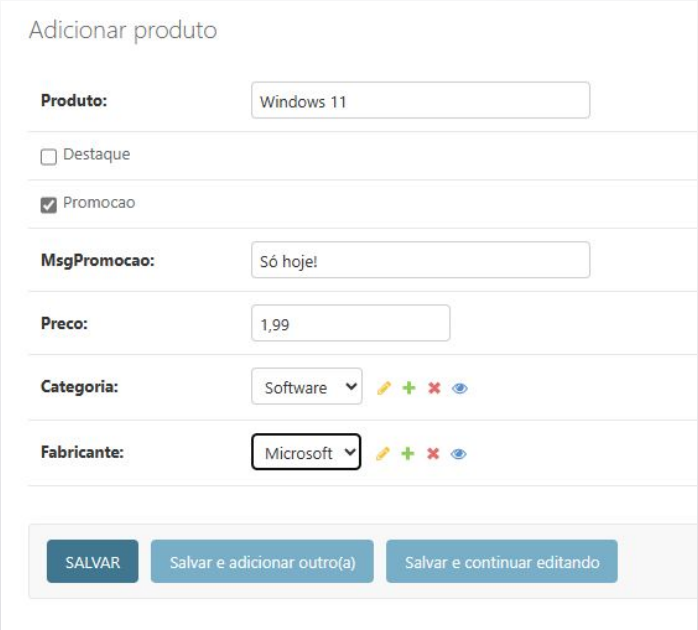
Iremos popular nosso banco de dados criando dois fabricantes(Microsoft e Dell), duas Categorias(Software e Hardware) e dois Produtos(Monitor associado a Del e Hardware, e Windows 11 associado a Microsoft e Software), para isso basta clicar em adicionar ou apertar na tabela(lista os registros), apertar em adicionar, preencher o formulário e enviar os dados.

Django
Capítulo 3 - Trabalhando com Models



Django
Capítulo 3 - Trabalhando com Models





Django
Capítulo 3 - Trabalhando com Models

Agora vamo acrescentar na tabela de produto um campo para imagem. Para isso, instalaremos a biblioteca Pillow a qual será usado pelo Django para os uploads de imagens. para instalar o Pillow basta ir no cmd e usar o comando:

```
pip install Pillow
```

Prompt de Comando

```
(venv) C:\Loja>pip install Pillow
Collecting Pillow
  Using cached pillow-10.2.0-cp311-cp311-win_amd64.whl (2.6 MB)
Installing collected packages: Pillow
Successfully installed Pillow-10.2.0

[notice] A new release of pip is available: 23.1.2 -> 23.3.2
[notice] To update, run: python.exe -m pip install --upgrade pip

(venv) C:\Loja>
```

Django
Capítulo 3 - Trabalhando com Models

Agora na classe modelo **Produto.py** adicione o campo para imagem:

```
#adicione abaixo do alterado_em = models.DateTimeField(auto_now=True)
image = models.ImageField(null=True, blank=True)
```

loja > models > Produto.py > Produto

```
1   from loja.models import *
2   class Produto(models.Model):
3       Produto = models.CharField(null=False, max_length=100)
4       destaque = models.BooleanField(default=True)
5       promocao = models.BooleanField(default=True)
6       msgPromocao = models.CharField(null=True, max_length=100)
7       preco = models.DecimalField(max_digits=8, decimal_places=2)
8       categoria = models.ForeignKey(Categoria, null=True, related_name='categoria', on_delete=models.SET_NULL)
9       fabricante = models.ForeignKey(Fabricante, null=True, related_name='fabricante', on_delete=models.SET_NULL)
10      criado_em = models.DateTimeField(auto_now_add=True)
11      alterado_em = models.DateTimeField(auto_now=True)
12      image = models.ImageField(null=True, blank=True)
13
14
15      def __str__(self):
16          return '{}'.format(self.Produto)
```

Django
Capítulo 3 - Trabalhando com Models

Antes de implementar essa modificação, adicione na lojaAdmin/settings/**settings.py** abaixo do “ STATIC_URL = 'static/' ”:

```
MEDIA_URL='/media/'
MEDIA_ROOT=os.path.join(BASE_DIR, 'media')
```

MEDIA_ROOT o caminho absoluto do sistema de arquivos para a pasta que conterá os arquivos enviados pelo usuário.

MEDIA_URL é quem lida com a mídia vinculado a MEDIA_ROOT, gerenciando seus arquivos

Django
Capítulo 3 - Trabalhando com Models

Agora no urls.py, na raiz do lojaAdmin, vamos mandor o Django liberar o acesso a pasta de imagem e o static. Para isso usamos esses imports no lojaAdmin/urls.py:

```
from django.conf.urls.static import static
from django.conf import settings
```

depois a somamos a pasta de imagens e o static à lista no urlpatterns, assim:

```
urlpatterns = [
            path('admin/', admin.site.urls),
    ] + static(settings.STATIC_URL, document_root=settings.STATIC_ROOT)
+ static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

Django
Capítulo 3 - Trabalhando com Models

settings.py:

```
117   # Follow link (ctrl + click) vaScript, Images)
118   # https://docs.djangoproject.com/en/4.2/howto/static-files/
119
120   STATIC_URL = 'static/'
121
122   MEDIA_URL='/media/'
123   MEDIA_ROOT=os.path.join(BASE_DIR, 'media')
```

urls.py:

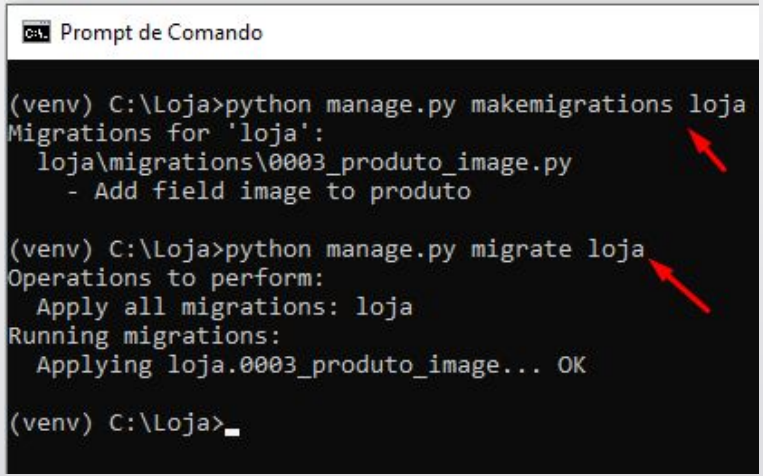
lojaAdmin > urls.py > ...

```
1  from django.contrib import admin
2  from django.urls import path
3  from django.conf.urls.static import static
4  from django.conf import settings
5
6
7  urlpatterns = [
8      path('admin/', admin.site.urls),
9  ]+ static(settings.STATIC_URL, document_root=settings.STATIC_ROOT) + static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

Django
Capítulo 3 - Trabalhando com Models

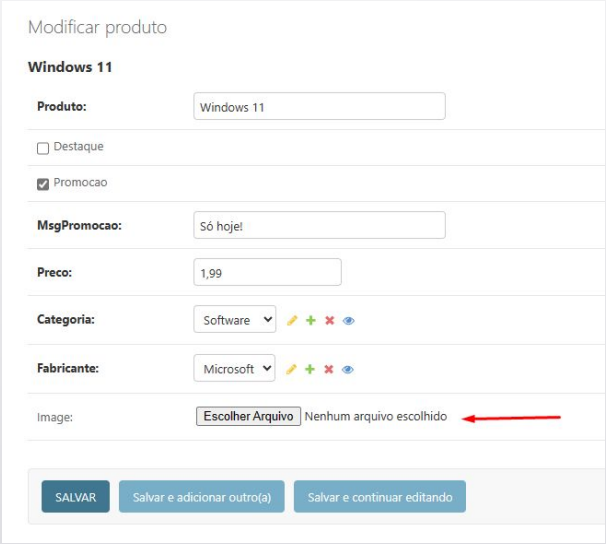
Agora vamos usar o comando para gerar o código da modificação, que dessa vez vai ser a alteração apenas de um campo, no banco de dados e depois o comando para implementar a modificação desse arquivo, novamente, no cmd:

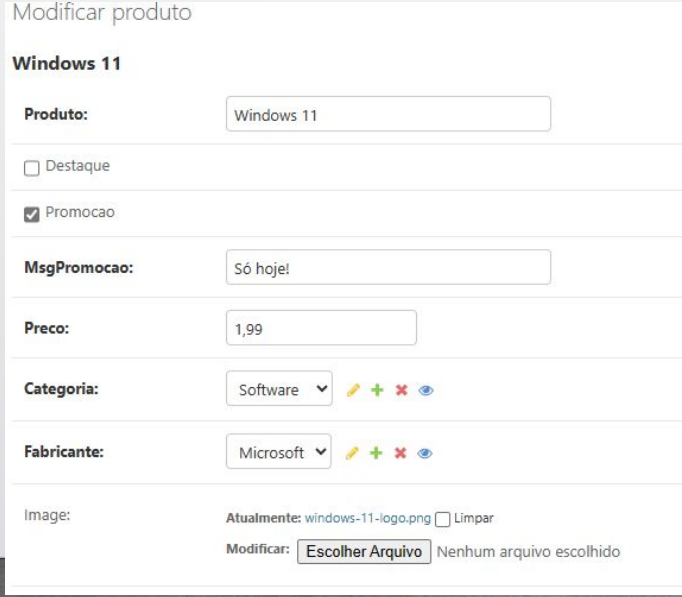
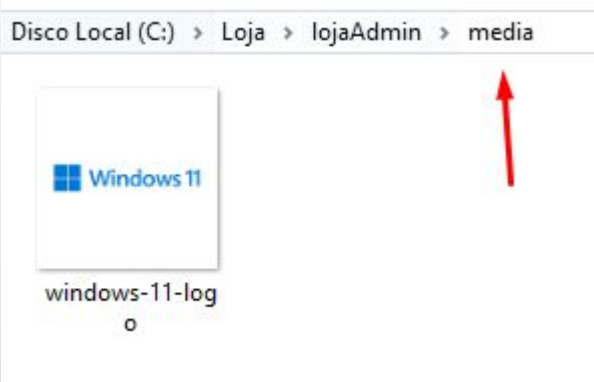
```
python manage.py makemigrations loja
python manage.py migrate loja
```



Django
Capítulo 3 - Trabalhando com Models

Rodando novamente a aplicação e clicando na tabela produto e selecionando um produto qualquer, veremos esse novo campo:



Django
Capítulo 3 - Trabalhando com Models

Quando adicionarmos uma imagem a pasta media será criada com a imagem que você submeteu